

Compiler Construction — Exam-Checklist Worksheet

Problems, hints & solutions for every point of the compilers_8020 checklist

Part IB — Cambridge CST — Paper 4

Work each problem before reading its Solution box (cover it with a sheet of paper). Warm-ups build the mechanics; the exam-style problem is the real thing.

1 Thompson's construction (regex $\rightarrow$ NFA)

Warm-up 1. Draw the Thompson NFA for the regex `ab`.

Hint. Concatenation links two sub-NFAs with an ϵ -edge from the first's accept to the second's start.

Solution

$\rightarrow (0) \xrightarrow{a} (1) \xrightarrow{\epsilon} (2) \xrightarrow{b} (3)^*$. Start 0, single accept 3.

Warm-up 2. Draw the Thompson NFA for `a|b`.

Hint. Alternation adds a fresh start with ϵ -edges into each branch, and a fresh accept that both branches ϵ -into.

Solution

Fresh start s ; $s \xrightarrow{\epsilon} (1) \xrightarrow{a} (2)$ and $s \xrightarrow{\epsilon} (3) \xrightarrow{b} (4)$; then $(2) \xrightarrow{\epsilon} f$, $(4) \xrightarrow{\epsilon} f$. Accept f .

Exam-style. Construct the Thompson NFA for `(a|b)*abb`.

Hint. Build `a|b` first, wrap it in the Kleene-star pattern (fresh start/accept, ϵ to enter the loop, ϵ to skip, ϵ back-edge), then concatenate `a`, `b`, `b`.

Solution

Let N be the `a|b` NFA (start s_1 , accept f_1). Star it: fresh s_0, f_0 with $s_0 \xrightarrow{\epsilon} s_1$, $s_0 \xrightarrow{\epsilon} f_0$ (skip), $f_1 \xrightarrow{\epsilon} s_1$ (loop), $f_1 \xrightarrow{\epsilon} f_0$. Then concatenate:

$$f_0 \xrightarrow{\epsilon} \bullet \xrightarrow{a} \bullet \xrightarrow{\epsilon} \bullet \xrightarrow{b} \bullet \xrightarrow{\epsilon} \bullet \xrightarrow{b} \text{(accept)}.$$

~ 11 states; many ϵ -edges. The construction is purely mechanical — one start, one accept, one operator at a time.

2 Subset construction (NFA $\rightarrow$ DFA with ϵ -closures)

Warm-up 1. For the NFA $0 \xrightarrow{\epsilon} 1$, $0 \xrightarrow{a} 2$, $1 \xrightarrow{b} 3$, compute $\epsilon\text{-closure}(\{0\})$.

Hint. Include the state itself plus everything reachable by ϵ -edges (transitively).

Solution

$\epsilon\text{-closure}(\{0\}) = \{0, 1\}$.

Warm-up 2. For that same NFA, compute $\text{move}(\{0, 1\}, a)$ then its ϵ -closure.

Hint. $\text{move}(S, x) = \text{states reachable from any state in } S \text{ on exactly one } x\text{-edge}$; then $\epsilon\text{-close}$.

Solution

$\text{move}(\{0, 1\}, a) = \{2\}; \epsilon\text{-closure}(\{2\}) = \{2\}.$

Exam-style. Run the subset construction on your $(a|b)^*abb$ NFA. Give the DFA states (as sets of NFA states) and the transition table; mark the accepting state.

Hint. The DFA start is $\epsilon\text{-closure}(\{\text{NFA start}\})$. Repeatedly: for each unmarked DFA state S and input x , $T = \epsilon\text{-closure}(\text{move}(S, x))$ is a (possibly new) DFA state. A DFA state accepts iff it contains an NFA accept state.

Solution

The well-known result is the 5-state DFA recognising “ends in **abb**”:

DFA state	on a	on b
A (start)	B	A
B (seen a)	B	C
C (seen ab)	B	D
D (seen abb) *	B	A

Each DFA state is the ϵ -closure of a set of NFA states. Note every state has a defined move on both inputs (because of the $*$ self-loop), and only D accepts. The classic “trap” is forgetting that on a mismatch you don’t go to a dead state — you fall back to the longest matching prefix (e.g. $C \xrightarrow{a} B$, not to start).

3 FIRST and FOLLOW sets

We use the standard expression grammar throughout items 3–4:

$$E \rightarrow TE' \quad E' \rightarrow +TE' \mid \epsilon \quad T \rightarrow FT' \quad T' \rightarrow *FT' \mid \epsilon \quad F \rightarrow (E) \mid \text{id}$$

Warm-up 1. Compute $\text{FIRST}(F)$ and $\text{FIRST}(T)$.

Hint. $\text{FIRST}(\alpha)$ is the set of terminals that can begin a string derived from α . For $T \rightarrow FT'$, $\text{FIRST}(T) = \text{FIRST}(F)$ since F can’t be empty.

Solution

$\text{FIRST}(F) = \{ (, \text{id} \}; \text{FIRST}(T) = \text{FIRST}(E) = \{ (, \text{id} \}.$

Warm-up 2. Compute $\text{FIRST}(E')$ and $\text{FIRST}(T')$.

Hint. These have an ϵ -production, so $\epsilon \in \text{FIRST}$.

Solution

$\text{FIRST}(E') = \{ +, \epsilon \}; \text{FIRST}(T') = \{ *, \epsilon \}.$

Exam-style. Compute the full FOLLOW sets for E, E', T, T', F .

Hint. Rules: $\$ \in \text{FOLLOW}(\text{start})$. For $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta) \setminus \{\epsilon\}$ to $\text{FOLLOW}(B)$; if $\beta \Rightarrow^* \epsilon$ (or β empty), add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

Solution

$\text{FOLLOW}(E) = \{\$, \,)\}$
 $\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{\$, \,)\}$
 $\text{FOLLOW}(T) = (\text{FIRST}(E') \setminus \{\epsilon\}) \cup \text{FOLLOW}(E) = \{+, \$, \,)\}$
 $\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{+, \$, \,)\}$
 $\text{FOLLOW}(F) = (\text{FIRST}(T') \setminus \{\epsilon\}) \cup \text{FOLLOW}(T) = \{*, +, \$, \,)\}$

4 LL(1) table; conflicts; running the parser

Warm-up 1. A grammar is LL(1) iff, for each non-terminal, the predict sets of its alternatives are disjoint. State the disjointness condition for $A \rightarrow \alpha \mid \beta$.

Hint. Two parts — one always, one only when an alternative is nullable.

Solution

$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$; and if (say) $\beta \Rightarrow^* \epsilon$ then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$.

Warm-up 2. Which table cell(s) get the production $E' \rightarrow +TE'$?

Hint. Put $A \rightarrow \gamma$ in $M[A, t]$ for each $t \in \text{FIRST}(\gamma)$.

Solution

$\text{FIRST}(+TE') = \{+\}$, so only $M[E', +] = E' \rightarrow +TE'$.

Exam-style. Build the LL(1) table for the expression grammar and trace parsing $\text{id} + \text{id} * \text{id}$.

Hint. For $A \rightarrow \alpha$: add to $M[A, t]$ for $t \in \text{FIRST}(\alpha)$; if $\epsilon \in \text{FIRST}(\alpha)$ also add for $t \in \text{FOLLOW}(A)$. Parse with an explicit stack: if top is a terminal, match; if a non-terminal, expand using M .

Solution

Key cells: $M[E, \text{id}] = M[E, (] = E \rightarrow TE'$; $M[E', +] = E' \rightarrow +TE'$; $M[E', \$] = M[E',)] = E' \rightarrow \epsilon$; $M[T, \text{id}] = M[T, (] = T \rightarrow FT'$; $M[T', *] = T' \rightarrow *FT'$; $M[T', +] = M[T', \$] = M[T',)] = T' \rightarrow \epsilon$; $M[F, \text{id}] = F \rightarrow \text{id}$, $M[F, (] = F \rightarrow (E)$.

Trace (stack top on the right; input $\text{id}+\text{id}*\text{id}\$$):

Stack	Action
$E\$$	expand $E \rightarrow TE'$
$E'T\$$	expand $T \rightarrow FT'$
$E'T'F\$$	expand $F \rightarrow \text{id}$, match id
$E'T'\$$	$T' \rightarrow \epsilon$ (lookahead $+$)
$E'\$$	$E' \rightarrow +TE'$, match $+$
$E'T\$$	$T \rightarrow FT'$, $F \rightarrow \text{id}$, match id
$E'T'\$$	$T' \rightarrow *FT'$, match $*$, $F \rightarrow \text{id}$, match id
$E'T'\$$	$T' \rightarrow \epsilon$, $E' \rightarrow \epsilon$
$\$$	accept

5 Eliminating left recursion & left-factoring

Warm-up 1. Eliminate the immediate left recursion in $A \rightarrow A\alpha \mid \beta$.

Hint. Introduce a tail non-terminal A' .

Solution

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \epsilon.$$

Warm-up 2. Left-factor $A \rightarrow ab \mid ac$.

Hint. Pull out the common prefix a .

Solution

$$A \rightarrow aA', \quad A' \rightarrow b \mid c.$$

Exam-style. Transform $E \rightarrow E + T \mid T$ to remove left recursion, and left-factor the statement grammar $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \text{other}$.

Hint. First problem: apply the $A \rightarrow A\alpha \mid \beta$ rule with $\alpha = +T$, $\beta = T$. Second: the two **if** alternatives share the prefix **if** E **then** S ; factor it and push the optional **else** into a new non-terminal (whose ϵ/else choice is the source of the dangling-else ambiguity).

Solution

Left recursion: $E \rightarrow TE', E' \rightarrow +TE' \mid \epsilon$.

Left-factoring:

$$S \rightarrow \text{if } E \text{ then } S S' \mid \text{other}, \quad S' \rightarrow \text{else } S \mid \epsilon.$$

Note this is now left-factored but still ambiguous: $M[S', \text{else}]$ has both $S' \rightarrow \text{else } S$ and (via FOLLOW) $S' \rightarrow \epsilon$ — the dangling-else conflict, conventionally resolved by preferring **else** (match to nearest **if**).

6 LR(0) item-set DFA (closure, GOTO, canonical collection)

Items 6–8 use the augmented grammar

$$S' \rightarrow S, \quad S \rightarrow AA, \quad A \rightarrow aA \mid b.$$

Warm-up 1. Compute $\text{closure}(\{S' \rightarrow \bullet S\})$.

Hint. If an item has a dot before non-terminal B , add $B \rightarrow \bullet \gamma$ for every production of B ; repeat to a fixpoint.

Solution

$$\{S' \rightarrow \bullet S, S \rightarrow \bullet AA, A \rightarrow \bullet aA, A \rightarrow \bullet b\}.$$

Warm-up 2. Compute $\text{GOTO}(I_0, a)$ where I_0 is that closure.

Hint. Advance the dot past every a in I_0 , then take the closure.

Solution

From $A \rightarrow \bullet aA$: $A \rightarrow a\bullet A$; closure adds $A \rightarrow \bullet aA$, $A \rightarrow \bullet b$. So $I_3 = \{A \rightarrow a\bullet A, A \rightarrow \bullet aA, A \rightarrow \bullet b\}$.

Exam-style. Build the full canonical collection of LR(0) item sets and the GOTO graph.

Hint. Start from I_0 , and for each state and each grammar symbol compute GOTO , creating new states as needed, until no new states appear.

Solution

I_0	$S' \rightarrow \bullet S, S \rightarrow \bullet AA, A \rightarrow \bullet aA, A \rightarrow \bullet b$	
I_1	$S' \rightarrow S \bullet$	(GOTO I_0, S)
I_2	$S \rightarrow A \bullet A, A \rightarrow \bullet aA, A \rightarrow \bullet b$	(GOTO I_0, A)
I_3	$A \rightarrow a \bullet A, A \rightarrow \bullet aA, A \rightarrow \bullet b$	(GOTO $I_0/I_2/I_3, a$)
I_4	$A \rightarrow b \bullet$	(GOTO on b)
I_5	$S \rightarrow AA \bullet$	(GOTO I_2, A)
I_6	$A \rightarrow aA \bullet$	(GOTO I_3, A)

GOTOs: $I_0: S \rightarrow I_1, A \rightarrow I_2, a \rightarrow I_3, b \rightarrow I_4$; $I_2: A \rightarrow I_5, a \rightarrow I_3, b \rightarrow I_4$; $I_3: A \rightarrow I_6, a \rightarrow I_3, b \rightarrow I_4$.

7 SLR(1) parse table & conflicts

Warm-up 1. For a complete item $A \rightarrow \alpha \bullet$ in state I_k , on which lookaheads do you place “reduce $A \rightarrow \alpha$ ”?

Hint. SLR uses FOLLOW.

Solution

On every terminal in FOLLOW(A) (and $A \rightarrow \alpha \bullet$ for the start production gives *accept* on \$).

Warm-up 2. In the ambiguous grammar $E \rightarrow E+E \mid \text{id}$, a state contains both $E \rightarrow E+E \bullet$ and $E \rightarrow E \bullet +E$. With $+$ as lookahead, what conflict arises?

Hint. One item says reduce, the other can shift.

Solution

A **shift/reduce conflict**: $+$ $\in$ FOLLOW(E) asks to *reduce* $E \rightarrow E+E$, while $E \rightarrow E \bullet +E$ asks to *shift* $+$. (This is left/right-associativity ambiguity; precedence declarations resolve it.)

Exam-style. Build the SLR(1) ACTION/GOTO table for the $S \rightarrow AA, A \rightarrow aA \mid b$ automaton; state whether it is conflict-free.

Hint. FOLLOW(S) = { $\$$ }, FOLLOW(A) = { $a, b, \$$ }. Shift terminals per GOTO; reduce complete items on their FOLLOW set; accept $S' \rightarrow S \bullet$ on \$.

Solution

state	a	b	\$	S	A
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r($A \rightarrow b$)	r($A \rightarrow b$)	r($A \rightarrow b$)		
5			r($S \rightarrow AA$)		
6	r($A \rightarrow aA$)	r($A \rightarrow aA$)	r($A \rightarrow aA$)		

Conflict-free — no cell holds two actions (this grammar is in fact LR(0); SLR’s FOLLOW restriction isn’t even needed here).

8 Running a shift-reduce parser

Warm-up 1. Configuration: stack 0 a 3, input b\$, using the table above. What is the next move?

Hint. Read ACTION[3, b].

Solution

ACTION[3, b] = s4: shift b, push state 4. Stack becomes 0 a 3 b 4.

Exam-style. Trace the shift-reduce parse of abb with the table above.

Hint. Shift pushes (symbol, state). Reduce $A \rightarrow \beta$ pops $2|\beta|$ entries, then pushes A and GOTO[exposed state, A].

Solution

Stack	Input	Action
0	abb\$	s3
0 a 3	bb\$	s4
0 a 3 b 4	b\$	reduce $A \rightarrow b$; GOTO(3,A)=6
0 a 3 A 6	b\$	reduce $A \rightarrow aA$; GOTO(0,A)=2
0 A 2	b\$	s4
0 A 2 b 4	\$	reduce $A \rightarrow b$; GOTO(2,A)=5
0 A 2 A 5	\$	reduce $S \rightarrow AA$; GOTO(0,S)=1
0 S 1	\$	accept

9 Translating to stack-machine bytecode

Assume a stack machine with PUSH n, LOAD x, STORE x, ADD/SUB/MUL, LT (pushes 1 if 2nd<top), CJUMP L (pop; jump to L if 0/false), JUMP L, CALL f, RET.

Warm-up 1. Translate $2 + 3 * 4$.

*Hint. Post-order: operands before operator; * binds tighter.*

Solution

PUSH 2; PUSH 3; PUSH 4; MUL; ADD (leaves 14 on the stack).

Warm-up 2. Translate the assignment $x = a + b$.

Hint. Evaluate the RHS onto the stack, then store.

Solution

LOAD a; LOAD b; ADD; STORE x.

Exam-style. Translate if $x < y$ then f(x) else g(y) to bytecode with labels.

Hint. Evaluate the condition, CJUMP to the else-label when false, emit the then-branch followed by a JUMP past the else-branch, then the else-branch under its label.

Solution

```

LOAD x
LOAD y
LT
CJUMP Lelse      ; if not (x<y), go to else
LOAD x
CALL f
JUMP Lend
Lelse: LOAD y
      CALL g
Lend:  ...      ; result of whichever branch on the stack

```

The two labels are the heart of it: one to skip into the else-branch, one to skip over it. Nested ifs just nest the same pattern with fresh labels.

10 Tail calls & TCO

Warm-up 1. In $f(x) = g(x) + 1$ and $h(x) = k(x)$, which call is a tail call?

Hint. A call is in tail position if its result is immediately returned with no pending work.

Solution

$k(x)$ in h is a tail call. $g(x)$ in f is not — the $+1$ is pending after it returns.

Exam-style. Define “tail call” and explain TCO at the machine level; rewrite `sum(n, acc)` for a list/range as tail-recursive and say why it now runs in constant stack space.

Hint. A non-tail recursion leaves a stack frame per call (work pending after the recursive call). TCO turns the call into a jump that reuses the current frame.

Solution

Tail call: a call that is the last action of its caller — the caller will do nothing with the result except return it, so nothing on the caller’s frame is needed afterwards.

TCO at machine level: instead of `CALL` (push return address + new frame) followed by `RET`, the compiler overwrites the current arguments in place and `JUMPS` to the callee’s entry, *reusing* the existing frame. Stack depth stays $O(1)$ instead of $O(n)$.

Tail-recursive sum:

```
sum(n, acc) = if n == 0 then acc else sum(n-1, acc+n)
```

The recursive call is the whole `else` (no pending + afterwards, because the addition is done *before* the call, in the argument). So each call replaces the current frame — constant stack. (Contrast `sum(n)=n+sum(n-1)`, where `+` is pending $\Rightarrow O(n)$ frames.)

11 Garbage collection schemes

Warm-up 1. Match each scheme to its mechanism: reference counting / mark-and-sweep / copying / generational.

Hint. Counts, reachability scan, two semispaces, age-based.

Solution

Reference counting — per-object count of incoming references, freed at 0. **Mark-and-sweep** — trace reachable objects from roots (mark), reclaim the rest (sweep). **Copying** — copy live objects from *from-space* to *to-space*, then flip. **Generational** — segregate by age; collect the young generation often, old rarely.

Warm-up 2. Why can’t reference counting reclaim a cycle?

Hint. Think of two objects pointing only at each other.

Solution

Mutually-referencing objects keep each other’s counts ≥ 1 even when unreachable from the roots, so their counts never hit 0. (Fix: a backup tracing collector, or weak references.)

Exam-style. Give one pro and one con of each of the four schemes, then recommend one for a workload with *many short-lived allocations and some cyclic data*, justifying your choice.

Hint. Lean on the weak generational hypothesis (most objects die young) and on which schemes handle cycles / fragmentation / pause times.

Solution

Scheme	Pro	Con
Reference counting	incremental, prompt reclaim	misses cycles; count overhead
Mark-and-sweep	collects cycles; no copying	fragmentation; stop-the-world pause
Copying	compacts (no fragmentation); fast alloc	needs 2× heap; copies live data
Generational	matches real lifetimes; short pauses	write barrier; cross-gen bookkeeping

Recommendation: **generational**, with a *copying* collector for the young generation. Short-lived objects die in the nursery, so most are reclaimed by a cheap, frequent copying collection of a small space; the rare survivors are promoted. A tracing-based design (unlike reference counting) reclaims the cyclic data correctly. This is exactly why production VMs (JVM, V8) use generational GC.

12 Exceptions implementation

Warm-up 1. What does “stack unwinding” mean?

Hint. What happens to the frames between the `throw` and the `catch`?

Solution

Popping the stack frames of the functions between the throw site and the matching handler — running any cleanup (destructors / **finally**) for each frame as it is discarded.

Exam-style. Describe how exceptions are implemented: handler frames, the exception pointer, and unwinding; then contrast with zero-cost table-based exceptions.

Hint. The simple scheme keeps a runtime linked list of active handlers; the modern scheme moves the cost off the common (no-exception) path using side tables.

Solution

Dynamic-registration scheme: entering a **try** pushes a *handler frame* (which exception types it catches + the handler address) onto a linked list; a global **exception pointer** points at the current (innermost) handler. **throw** walks this list from the exception pointer outwards: for each frame it *unwinds* (pops the frame, running destructors/finally) until it finds a handler whose type matches, then transfers control there. Cost is paid on *every try* entry/exit, even when nothing is thrown.

Zero-cost (table-based) scheme: no runtime registration. The compiler emits static **side tables** mapping each code range (PC) to its handlers and cleanup actions. The happy path costs *nothing*; only on a **throw** does the runtime consult the tables and unwind. Trade-off: larger binary, slower throw. Modern C++/Rust use this.

13 Standard optimisations

Warm-up 1. Give the example transformation for *constant folding* and *strength reduction*.

Hint. Evaluate at compile time; replace a costly op with a cheap one.

Solution

Constant folding: $2 + 3 \rightarrow 5$. Strength reduction: $x * 2 \rightarrow x \ll 1$.

Warm-up 2. What enabling condition must hold for $x * 0 \rightarrow 0$?

Hint. Think about side effects in evaluating x .

Solution

Evaluating `x` must have *no side effects* (and not trap); otherwise discarding it changes behaviour.

Exam-style. Optimise the following, naming each transformation you apply, and give the final code:

```
t = 4 * 2;
y = x + t;
z = x + t;
unused = z * 0;
for i in 0..n: w = a * b + i;
```

Hint. Fold the constant, propagate it, eliminate the dead store, share the repeated subexpression, and hoist the loop-invariant.

Solution

1. **Constant folding:** `t = 4*2` $\rightarrow$ `t = 8`.
2. **Constant propagation:** `y = x + 8`, `z = x + 8`.
3. **Common-subexpression elimination:** compute `x + 8` once: `c = x + 8`; `y = c`; `z = c`.
4. **Dead-code elimination:** `unused = z * 0` (and `unused`) is never used $\rightarrow$ remove.
5. **Loop-invariant code motion:** `a * b` doesn't depend on `i` $\rightarrow$ hoist.

Final:

```
c = x + 8;  y = c;  z = c;
m = a * b;
for i in 0..n: w = m + i;
```

(If `y/z` themselves turn out unused, a further DCE pass removes them too.)

That's every checklist point. Re-do the exam-style ones cold a week before the paper.